```
      @source[field].value = fieldResponse.value
    else
      @addField field, fieldResponse.value
```

The advantage of a `Proxy` approach is that it provides a way to unify an interface:

```
form = new Formulaic document, '#form', http,
'http://agtron.co/formulaic/1'

form.fields.login = 'scruffy1234'
```

You *only* change properties on an instance of the `Formulaic` class, but the underlying implementation can be communicating with the server and updating the view for you. In fact, the proxy can do *anything* in response to a `get` or `set`. Listing 13.5 shows one other use for proxies, by validating the field value to ensure empty strings are not used.

The fun doesn't stop at proxies, though. Another exciting feature proposed for ECMAScript 6 is the addition of iterators, generators, and the concept of `yield`, which combine to finally give comprehensions real power.

### 13.3.5. Comprehensions, iterators, and generators

Comprehensions, iterators, and generators in ECMAScript 6 will be familiar to Python programmers. Why do they matter? Event streams. You'll remember (from chapter 9) that composing programs with event streams is problematic. To improve these programs, you developed a fluent interface for asynchronous streams of events that abstracted away some of the complexity, giving you a cleaner programming model to work with. The combination of comprehensions, iterators, and generators can provide the same power at a different syntactic level.

First, though, you need to see how JavaScript is getting comprehensions very much like the ones you already know in CoffeeScript.

#### Comprehensions